

Kompresija slike algoritmom brze Furijeove transformacije

Profesor: prof. dr Filip Marić

Asistent: Milan Mitreski

Student: David Dedić

Beograd, maj 2026

Sadržaj

1.	Uvod	2
2.	Teorijska osnova	3
2.1	Furijev red	3
2.2	Diskretna Furijeova transformacija	4
2.3	Brza Furijeova transformacija	5
2.4	Dvodimenzionalna diskretna Furijeova transformacija	6
2.5	Opis algoritma	6
3.	Implementacija kompresije slike u C++-u	9
3.1	Biblioteka za kompleksne brojeve	9
3.2	Biblioteka za FFT	12
3.3	Objedinjavanje komponenti algoritma	21
4.	Prednosti i ograničenja implementiranog algoritma	25

Uvod

Ne može se zanemariti važnost digitalnih slika u današnjem svijetu. Sa porastom rezolucije i kvaliteta slika raste i količina memorije potrebna za njihovo čuvanje, kao i vrijeme potrebno za njihovo slanje preko mejla, društvenih mreža itd. Zbog toga kompresija predstavlja važan problem u obradi digitalnih slika.

Osnovni cilj kompresije bilo kojeg podatka je da se smanji količina memorije potrebnih za njegovo čuvanje, uz zadržavanje prihvatljivog kvaliteta. Kompresija može biti bez gubitaka (engl. **loseless compression**), kada se originalna slika može potpuno rekonstruisati, ili sa gubicima (engl. **lossy compression**), kada se dio informacija namjerno odbacuje kako bi se postigao veći stepen kompresije. U ovom radu razmatramo kompresiju slike sa gubicima zasnovanoj na frekvencijskoj analizi slike.

Slika u računarima se predstavlja kao matrica piksela. Kod slika u boji svaki piksel nosi se predstavlja pomoću 3 komponente: crvene, zelene i plave (brojevi od 0 do 255). Umjesto direktne obrade piksela, moguće je sliku transformisati iz prostornog u frekvencijski domen korišćenjem 2D brze Furijeove transformacije. U frekvencijskom domenu slika se opisuje pomoću koeficijenata koji predstavljaju zastupljenost određenih frekvencijskih komponenti.

U ovom radu za transformaciju slike koristi se dvodimenzionalna brza Furijeova transformacija (2D FFT). Nad svakim RGB kanalom posebno se primjenjuje 2D FFT, nakon čega se zadržava određeni procenat koeficijenata najveće amplitude, dok se ostali koeficijenti postavljaju na nulu. Na taj način se odbacuju manje značajne frekvencijske komponente. Nakon toga se primjenjuje inverzna brza Furijeova transformacija, čime se dobija rekonstruisana slika.

Teorijska osnova i opis algoritma

2.1 Furijeov red

Furijska analiza, a i obrada signala se zasniva na ideji da se funkcija (signal) može razložiti na zbir jednostavnih periodičnih funkcija koje se dalje obrađuju.

Neka je f kompleksno-vrijednosna periodična funkcija perioda 2π . Takve funkcije posmatraju se u **Hilbertovom** prostoru

$$L^2([-\pi, \pi]),$$

odnosno prostoru kvadratno integrabilnih funkcija na intervalu $[-\pi, \pi]$. Na ovom prostoru definiše se skalarni proizvod

$$\langle f, g \rangle = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) \overline{g(x)} dx.$$

U ovom prostoru funkcije

$$e_n(x) = e^{inx}, \quad n \in \mathbb{Z},$$

čine ortonormirani sistem.

$$\langle e_m, e_n \rangle = \begin{cases} 1, & m = n, \\ 0, & m \neq n. \end{cases}$$

Osim toga, ove funkcije čine i potpun sistem u prostoru $L^2([-\pi, \pi])$, što znači da su kvadratno integrabilne funkcije jednake svom **Furijskom redu** u srednjekvadratnom smislu tj.

$$f(x) = \sum_{n=-\infty}^{\infty} c_n e^{inx},$$

gdje su koeficijenti c_n dati formulom

$$c_n = \langle f, e_n \rangle = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) e^{-inx} dx.$$

Jedan od značajnih rezultata Furijeove analize je **Parsevalova jednakost**.

Teorema 1: Neka je f kvadratno-integrabilna funkcija u prostoru $L^2([-\pi, \pi])$ i $c_n = \langle f, e_n \rangle$. Tada je:

$$\|f\|^2 = \sum_{n=-\infty}^{\infty} |c_n|^2.$$

Ona pokazuje da se ukupna "energija" funkcije može izraziti preko modula njenih Furijeovih koeficijenata. Zbog toga koeficijenti većeg modula imaju veći doprinos funkciji, dok koeficijenti manjeg modula imaju manji uticaj. Ovakva osobina Furijeove transformacije nam zapravo omogućava da uradimo kompresiju slike pomoću nje.

2.2 Diskretna Furijeova transformacija

Neka je dat konačan niz kompleksnih brojeva

$$x_0, x_1, \dots, x_{N-1}.$$

Ovaj niz možemo posmatrati kao diskretni signal dužine N . Diskretna Furijeova transformacija, skraćeno DFT, ovom nizu pridružuje novi niz kompleksnih brojeva

$$X_0, X_1, \dots, X_{N-1},$$

definisan formulom

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}, \quad k = 0, 1, \dots, N-1.$$

Brojevi X_k nazivaju se Furijeovi koeficijenti diskretnog signala. Svaki koeficijent opisuje koliko je frekvencija k prisutna u datom signalu. Pored frekvencijske interpretacije, DFT ima i algebarsku interpretaciju. Ako se vrijednosti diskretnog signala posmatraju kao koeficijenti polinoma $P(z)$, tada DFT računa vrijednosti tog polinoma u N -tim korijenima jedinice. Pošto postoji N koeficijenata, direktno računanje DFT-a nekog vektora zahtijeva približno N^2 operacija. Zbog toga je vremenska složenost direktnog DFT algoritma $O(N^2)$.

Vremenska složenost direktnog računanja DFT-a nije pogodna za kompresiju jer su slike često sastavljene od miliona piksela na što kada primijenimo algoritam kvadratne složenosti postane preveliki broj operacija.

2.3 Brza Furijeova transformacija

Brza Furijeova transformacija je algoritam koji računa DFT nekog vektora u $O(N \log N)$ operacija. U ovom seminarskom radu akcenat smo stavili na Cooley-Tukey algoritmu za računanje FFT-a koji radi za nizove čija je dužina stepen dvojke. Nego tada, lako možemo preći na računanje FFT-a vektora proizvoljne dužine tako što ćemo dopuniti početni vektor nulama (na kraju ili na početku u zavisnosti od definicije DFT-a). Uvedimo oznaku

$$\omega_N = e^{-2\pi i/N}.$$

Tada se DFT može zapisati u obliku

$$X_k = \sum_{n=0}^{N-1} x_n \omega_N^{kn}.$$

Ugrubo, ideja je da je FFT jednočlanog niza sam taj niz. Rekursivno izračunamo FFT prve i druge polovine niza i na kraju FFT početnog niza izračunamo formulama:

$$X_k = E_k + \omega_N^k O_k, \quad k = 0, 1, \dots, \frac{N}{2} - 1,$$

$$X_{k+N/2} = E_k - \omega_N^k O_k, \quad k = 0, 1, \dots, \frac{N}{2} - 1.$$

Jasno je da korišćenje FFT-a značajno popravljja vrijeme obrade većih podataka poput slika, što je čini mnogo pogodnijom od direktnog računanja DFT-a za kompresiju, čemu će biti posvećena posebna analiza kasnije.

Osim obrade signala, algoritam brze Furijeove transformacije ima i značajnu primjenu u kriptografiji jer nam omogućava da množimo polinome u $O(N \log N)$ operacija. Interesantno je da je dugo vremena naučna zajednica vjerovala da ne postoji efikasniji algoritam za množenje polinoma od kvadratnog, što je opovrgnuto u dva navrata sa pojavom Karacubinovog algoritma za množenje polinoma i NTT (**N**umber **t**heoretic **t**ransform).

2.4 2D FFT

Prethodna razmatranja za jednu dimenziju se daju lako generalizovati za više dimenzija tako što se umjesto prostora $L^2([-\pi, \pi])$ posmatra $L^2([-\pi, \pi] \times [-\pi, \pi] \times \dots \times [-\pi, \pi])$, gdje broj intervala u Dekartovom proizvodu odgovara dimenziji prostora. Pošto posmatramo slike koje su u računarima predstavljene kao matrice piksela, posebno će nam vrijedno biti razmatranje dvodimenzionalne brze Furijeove transformacije.

Dvodimenzionalna diskretna Furijeova transformacija matrice $X = (x_{ij})$ definiše se formulom

$$F_{uv} = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} x_{ij} e^{-2\pi i \left(\frac{ui}{M} + \frac{vj}{N} \right)},$$

gdje su

$$u = 0, 1, \dots, M-1, \quad v = 0, 1, \dots, N-1.$$

Koeficijent F_{uv} predstavlja zastupljenost dvodimenzionalne frekvencijske komponente određene parom (u, v) . Dalje, 2D FFT se radi tako što se primijeni FFT nad vrstama matrice, a zatim nad kolonama. Takođe, mogao bi prvo i da se primijeni FFT nad kolonama matrice, a zatim nad vrstama. Pokaže se da se u oba slučaja dobije jedno te ista rezultujuća matrica.

2.5 Opis algoritma

U ovom radu implementiran je algoritam kompresije slike u boji pomoću dvodimenzionalne brze Furijeove transformacije. Slika se učitava u RGB formatu, zatim se razdvaja na tri kanala: crveni, zeleni i plavi. Nad svakim kanalom posebno se primjenjuje isti postupak kompresije.

Pošto implementirana verzija FFT algoritma koristi radix-2 Cooley–Tukey pristup, dimenzije signala moraju biti stepeni broja 2. Zbog toga se, ukoliko dimenzije slike nisu oblika $2^m \times 2^k$, svaki kanal dopunjava nulama do najbližih većih dimenzija koje jesu stepeni broja 2. Nakon rekonstrukcije, dodatni dio slike se odsijeca i rezultat se vraća na originalne dimenzije.

Ulazna slika se učitava pomoću biblioteke `stb_image`. Nakon učitavanja, slika je predstavljena kao niz piksela u obliku

$$R, G, B, R, G, B, \dots$$

gdje svaka trojka vrijednosti predstavlja jedan piksel slike. Zatim se slika razdvaja na tri odvojene matrice, odnosno tri kanala:

$$R = (r_{ij}), \quad G = (g_{ij}), \quad B = (b_{ij}).$$

Svaki od ovih kanala predstavlja jednu matricu dimenzija $M \times N$, gdje su M i N visina i širina slike. Vrijednosti elemenata matrice su intenziteti odgovarajuće boje u pikselima.

Nakon dopunjavanja dimenzija, nad svakim kanalom se primjenjuje dvodimenzionalna FFT. Dvodimenzionalna FFT se računa tako što se prvo primijeni jednodimenzionalna FFT na svaku vrstu matrice, a zatim jednodimenzionalna FFT na svaku kolonu dobijene matrice.

Za svaki kanal dobija se matrica kompleksnih koeficijenata:

$$F_{uv}^R, \quad F_{uv}^G, \quad F_{uv}^B.$$

Ovi koeficijenti predstavljaju frekvencijski sadržaj odgovarajućih RGB kanala. Koeficijenti većeg modula imaju veći uticaj na izgled slike, dok koeficijenti manjeg modula najčešće predstavljaju sitne detalje ili manje izražene promjene.

Kompresija se vrši odbacivanjem dijela FFT koeficijenata. Za svaki koeficijent računa se njegov modul:

$$|F_{uv}| = \sqrt{\text{Re}(F_{uv})^2 + \text{Im}(F_{uv})^2}.$$

Zatim se zadržava samo određeni procenat koeficijenata najvećeg modula, dok se ostali koeficijenti postavljaju na nulu. Na primjer, ako se zadržava 10% koeficijenata, tada se odbacuje 90% frekvencijskih komponenti.

Ovaj postupak se posebno primjenjuje na svaki od RGB kanala. Time se smanjuje količina frekvencijskih informacija koje se koriste za rekonstrukciju slike. Nakon filtriranja koeficijenata primjenjuje se inverzna dvodimenzionalna FFT. Inverznom transformacijom svaki RGB kanal se vraća iz frekvencijskog domena u prostorni domen. Pošto su neki koeficijenti prethodno

postavljeni na nulu, rekonstruisani kanal nije identičan originalnom, već predstavlja njegovu aproksimaciju.

Nakon inverzne transformacije, vrijednosti piksela se ograničavaju na interval

$$[0, 255],$$

jer RGB vrijednosti moraju pripadati tom opsegu. Zatim se rekonstruisani kanali R , G i B spajaju u jednu sliku u boji. Rekonstruisana slika se čuva pomoću biblioteke `stb_image_write`. Izlazna slika ima iste dimenzije kao originalna slika. Ukoliko je tokom obrade korišteno dopunjavanje nulama, dodatni redovi i kolone se nakon inverzne FFT odsijecaju, tako da konačni rezultat odgovara originalnim dimenzijama slike.

Implementacija kompresije slike u C++-u

Implementiraćemo najprije biblioteku za rad s kompleksnim brojevima, a zatim i biblioteku za računanje brze Furijeove transformacije. To će nam biti dovoljno da bismo implementirali kompresiju slike.

Biblioteka za kompleksne brojeve

Listing 1: Fajl complex.hpp

```
1 #ifndef COMPLEX_HPP
2 #define COMPLEX_HPP
3
4 #include <iostream>
5
6
7 class Complex{
8 public:
9     Complex(double real = 0.0, double imaginary = 0.0)
10         : _real(real), _imaginary(imaginary) {}
11
12     double real_part() const;
13     double imaginary_part() const;
14     void set_real_part(double real);
15     void set_imaginary_part(double imaginary);
16     double magnitude() const;
17
18     Complex operator+(const Complex& z) const;
19     Complex operator-(const Complex& z) const;
20     Complex operator*(const Complex& z) const;
21     Complex operator/(const Complex& z) const;
22     Complex conjugate() const;
23
24     bool operator==(const Complex& z) const;
25     bool operator<(const Complex& z) const;
26
27     Complex exp() const;
28 private:
29     double _real;
```

```

30     double _imaginary;
31 };
32
33 std::ostream& operator<<(std::ostream& output, const
    Complex& z);
34 std::istream& operator>>(std::istream& input, Complex& z
    );
35
36 #endif

```

Listing 2: Fajl complex.cpp

```

1  #include <iostream>
2  #include "complex.hpp"
3  #include <cmath>
4  #include <stdexcept>
5
6  double Complex::real_part() const{
7      return _real;
8  }
9
10 double Complex::imaginary_part() const{
11     return _imaginary;
12 }
13
14 void Complex::set_real_part(double real){
15     _real = real;
16 }
17
18 void Complex::set_imaginary_part(double imaginary){
19     _imaginary = imaginary;
20 }
21
22 double Complex::magnitude() const{
23     return std::sqrt(_real*_real + _imaginary*_imaginary
        );
24 }
25
26 Complex Complex::operator+(const Complex& z) const{
27     return Complex(_real + z.real_part(), _imaginary + z
        .imaginary_part());

```

```

28 }
29
30 Complex Complex::operator-(const Complex& z) const{
31     return Complex(_real - z.real_part(), _imaginary - z
32         .imaginary_part());
33 }
34
35 Complex Complex::operator*(const Complex& z) const{
36     return Complex(_real * z.real_part() - _imaginary *
37         z.imaginary_part(), _real * z.imaginary_part() +
38         _imaginary * z.real_part());
39 }
40
41 Complex Complex::operator/(const Complex& z) const{
42     if(z.real_part() == 0.0 && z.imaginary_part() ==
43         0.0)
44         throw std::runtime_error("Division by zero");
45
46     Complex z0 = (*this)*(z.conjugate());
47     z0.set_real_part(z0.real_part()/(z.real_part() * z.
48         real_part() + z.imaginary_part() *z.
49         imaginary_part()));
50     z0.set_imaginary_part(z0.imaginary_part()/(z.
51         real_part() * z.real_part() + z.imaginary_part()
52         *z.imaginary_part()));
53
54     return z0;
55 }
56
57 Complex Complex::conjugate() const{
58     return Complex(_real, -_imaginary);
59 }
60
61 bool Complex::operator==(const Complex& z) const{
62     return _real == z.real_part() && _imaginary == z.
63         imaginary_part();
64 }
65
66 bool Complex::operator<(const Complex& z) const{
67     return this->magnitude() < z.magnitude();
68 }

```

```

59 }
60
61 Complex Complex::exp() const{
62     double e = std::exp(_real);
63
64     return Complex(
65         e * std::cos(_imaginary),
66         e * std::sin(_imaginary)
67     );
68 }
69
70 std::ostream& operator<<(std::ostream& output, const
    Complex& z){
71     output << z.real_part();
72
73     if (z.imaginary_part() >= 0)
74         output << "+" << z.imaginary_part() << "i";
75     else
76         output << z.imaginary_part() << "i";
77
78     return output;
79 }
80
81
82 std::istream& operator>>(std::istream& input, Complex& z
    ){
83     double real,imaginary;
84     input >> real >> imaginary;
85
86     z.set_real_part(real);
87     z.set_imaginary_part(imaginary);
88
89     return input;
90 }

```

Biblioteka za FFT

Listing 3: Fajl fft.hpp

```

1 #ifndef FFT_HPP
2 #define FFT_HPP

```

```

3 #include <iostream>
4 #include <vector>
5 #include "complex.hpp"
6
7 namespace FFT{
8     bool isPowerOfTwo(int n);
9     int nextPowerOfTwo(int n);
10
11     void fft(const std::vector<Complex>& a, int start_a,
12             const std::vector<Complex>& w,
13             std::vector<Complex>& rezultat, int
14             start_rezultat, int korak);
15     std::vector<Complex> fft(const std::vector<Complex>&
16                             a);
17     std::vector<Complex> ifft(const std::vector<Complex>
18                             &a);
19     std::vector<Complex> fftPadded(const std::vector<
20                                 Complex>& a);
21     std::vector<Complex> ifftPadded(const std::vector<
22                                 Complex> &a, int originalSize);
23
24     std::vector<Complex> fft2D(const std::vector<Complex>
25                               &data, int width, int height);
26     std::vector<Complex> ifft2D(const std::vector<
27                                Complex>& data, int width, int height);
28     std::vector<Complex> fft2DPadded(const std::vector<
29                                    Complex>& data, int width, int height, int
30                                    paddedWidth, int paddedHeight);
31     std::vector<Complex> ifft2DPadded(const std::vector<
32                                     Complex>& data, int originalWidth, int
33                                     originalHeight, int paddedWidth, int paddedHeight
34                                     );
35 }
36
37 #endif

```

Listing 4: Fajl fft.cpp

```

1     #include "fft.hpp"
2 #include <iostream>
3 #include <vector>

```

```

4 #include <cmath>
5 #include <stdexcept>
6
7 bool FFT::isPowerOfTwo(int n){
8     return n > 0 && (n & (n-1)) == 0;
9 }
10
11 int FFT::nextPowerOfTwo(int n){
12     int power = 1;
13
14     while(n > power){
15         power *= 2;
16     }
17
18     return power;
19 }
20
21 void FFT::fft(const std::vector<Complex>& a, int start_a
22     , const std::vector<Complex>& w,
23     std::vector<Complex>& rezultat, int
24     start_rezultat, int korak){
25
26     int n = a.size()/korak;
27
28     if(n == 1){
29         rezultat[start_rezultat] = a[start_a];
30         return;
31     }
32
33     fft(a, start_a, w, rezultat, start_rezultat, korak*2);
34     fft(a, start_a + korak, w, rezultat, start_rezultat + n
35         /2, korak* 2);
36
37     for(int k = 0; k < n/2; k++){
38         Complex r1 = rezultat[start_rezultat + k];
39         Complex r2 = rezultat[start_rezultat + (k + n/2)
40             ];
41
42         rezultat[start_rezultat + k] = r1 + w[k*korak]*
43             r2;

```

```

39         rezultat[start_rezultat + (k + n/2)] = r1 - w[k*
           korak]*r2;
40     }
41 }
42
43 std::vector<Complex> FFT::fft(const std::vector<Complex
    >& a){
44     int n = a.size();
45
46     if (!isPowerOfTwo(n)) {
47         throw std::runtime_error("FFT zahtijeva duzinu
           koja je stepen dvojke.");
48     }
49
50     std::vector<Complex> w(n/2);
51
52     for(int k = 0; k < n/2; k++){
53         w[k] = (Complex(2 * k * M_PI/n, 0) * Complex
           (0,1)).exp();
54     }
55
56     std::vector<Complex> rezultat(n);
57     fft(a,0,w,rezultat,0,1);
58
59     return rezultat;
60 }
61
62 std::vector<Complex> FFT::ifft(const std::vector<Complex
    >& a){
63     int n = a.size();
64
65     if (!isPowerOfTwo(n)) {
66         throw std::runtime_error("IFFT zahtijeva duzinu
           koja je stepen dvojke.");
67     }
68
69     std::vector<Complex> w(n);
70     for(int k = 0; k < n; k++){
71         w[k] = (Complex(-2 * k * M_PI/n, 0) * Complex
           (0,1)).exp();

```



```

72
73     std::vector<Complex> rezultat(n);
74
75     fft(a,0,w,rezultat,0,1);
76
77     for(int i = 0; i < n; i++)
78         rezultat[i] = rezultat[i]/Complex(n,0);
79
80     return rezultat;
81 }
82
83 std::vector<Complex> FFT::fftPadded(const std::vector<
Complex>& a){
84     if(a.empty())
85         throw std::runtime_error("FFT ne moze nad
praznim nizom.");
86
87     int n = a.size();
88
89     if(isPowerOfTwo(n))
90         return fft(a);
91
92     int paddedSize = nextPowerOfTwo(n);
93
94     std::vector<Complex> b(paddedSize);
95
96     for(int j = 0; j < n; j++)
97         b[j] = a[j];
98
99     return fft(b);
100 }
101
102 std::vector<Complex> FFT::ifftPadded(
103     const std::vector<Complex>& a,
104     int originalSize
105 ) {
106     if (a.empty()) {
107         throw std::runtime_error("IFFT ne moze raditi
nad praznim nizom.");
108     }

```

```

109
110     if (originalSize <= 0 || originalSize > (int)a.size
111         ()) {
112         throw std::runtime_error("Neispravna originalna
113             duzina.");
114     }
115
116     std::vector<Complex> reconstructed = FFT::ifft(a);
117
118     reconstructed.resize(originalSize);
119
120     return reconstructed;
121 }
122
123 std::vector<Complex> FFT::fft2D(const std::vector<
124     Complex>& data, int width, int height){
125     if((int) data.size() != width * height){
126         throw std::runtime_error("Neispravna velicina
127             podataka za 2D FFT.");
128     }
129
130     std::vector<Complex> result = data;
131
132     for(int y = 0; y < height; y++){
133         std::vector<Complex> row(width);
134
135         for(int x= 0; x < width; x++){
136             row[x] = result[y * width + x];
137         }
138
139         row = FFT::fft(row);
140
141         for(int x = 0; x < width; x++){
142             result[y * width + x] = row[x];
143         }
144     }
145
146     for(int x = 0; x < width; x++){
147         std::vector<Complex> column(height);

```

```

145
146     for(int y = 0; y < height; y++){
147         column[y] = result[y * width + x];
148     }
149
150     column = FFT::fft(column);
151
152     for(int y = 0; y < height; y++){
153         result[y * width + x] = column[y];
154     }
155 }
156
157 return result;
158 }
159
160 std::vector<Complex> FFT::ifft2D(const std::vector<
    Complex>& data, int width, int height){
161     if((int) data.size() != width * height){
162         throw std::runtime_error("Neispravna velicina
            podataka za 2D IFFT.");
163     }
164
165     std::vector<Complex> result = data;
166
167     for(int y = 0; y < height; y++){
168         std::vector<Complex> row(width);
169
170         for(int x = 0; x < width; x++){
171             row[x] = result[y * width + x];
172         }
173
174         row = FFT::ifft(row);
175
176         for(int x = 0; x < width; x++){
177             result[y * width + x] = row[x];
178         }
179     }
180
181     for(int x = 0 ; x < width; x++){
182         std::vector<Complex> column(height);

```

```

183
184     for(int y = 0; y < height; y++){
185         column[y] = result[y*width + x];
186     }
187
188     column = FFT::ifft(column);
189
190     for(int y = 0; y < height; y++){
191         result[y * width + x] = column[y];
192     }
193 }
194
195 return result;
196 }
197
198 std::vector<Complex> FFT::fft2DPadded(
199     const std::vector<Complex>& data,
200     int width,
201     int height,
202     int paddedWidth,
203     int paddedHeight
204 ) {
205     if ((int)data.size() != width * height) {
206         throw std::runtime_error("Neispravna velicina
207             podataka za 2D FFT.");
208     }
209
210     if (paddedWidth < width || paddedHeight < height) {
211         throw std::runtime_error("Padovane dimenzije
212             moraju biti vece ili jednake originalnim.");
213     }
214
215     if (!isPowerOfTwo(paddedWidth) || !isPowerOfTwo(
216         paddedHeight)) {
217         throw std::runtime_error("Padovane dimenzije
218             moraju biti stepeni dvojke.");
219     }
220
221     std::vector<Complex> padded(
222         paddedWidth * paddedHeight,

```

```

219         Complex(0.0, 0.0)
220     );
221
222     for (int y = 0; y < height; y++) {
223         for (int x = 0; x < width; x++) {
224             padded[y * paddedWidth + x] = data[y * width
225                 + x];
226         }
227     }
228
229     return FFT::fft2D(padded, paddedWidth, paddedHeight)
230         ;
231 }
232
233 std::vector<Complex> FFT::ifft2DPadded(
234     const std::vector<Complex>& data,
235     int originalWidth,
236     int originalHeight,
237     int paddedWidth,
238     int paddedHeight
239 ) {
240     if ((int)data.size() != paddedWidth * paddedHeight)
241     {
242         throw std::runtime_error("Neispravna velicina
243             podataka za 2D IFFT.");
244     }
245
246     if (paddedWidth < originalWidth || paddedHeight <
247         originalHeight) {
248         throw std::runtime_error("Padovane dimenzije
249             moraju biti vece ili jednake originalnim.");
250     }
251
252     if (!isPowerOfTwo(paddedWidth) || !isPowerOfTwo(
253         paddedHeight)) {
254         throw std::runtime_error("Padovane dimenzije
255             moraju biti stepeni dvojke.");
256     }
257
258     std::vector<Complex> reconstructed =

```

```

251         FFT::ifft2D(data, paddedWidth, paddedHeight);
252
253         std::vector<Complex> cropped(originalWidth *
                originalHeight);
254
255         for (int y = 0; y < originalHeight; y++) {
256             for (int x = 0; x < originalWidth; x++) {
257                 cropped[y * originalWidth + x] =
258                     reconstructed[y * paddedWidth + x];
259             }
260         }
261
262         return cropped;
263     }

```

Objedinjavanje komponenti algoritma

Listing 5: Fajl main.cpp

```

1  #include <iostream>
2  #define STB_IMAGE_IMPLEMENTATION
3  #include "stb_image.h"
4  #define STB_IMAGE_WRITE_IMPLEMENTATION
5  #include "stb_image_write.h"
6  #include <vector>
7  #include "complex.hpp"
8  #include "fft.hpp"
9  #include <algorithm>
10
11 void keepLargestCoefficients(std::vector<Complex>& F,
        double keepRatio){
12     std::vector<double> magnitudes;
13     magnitudes.reserve(F.size());
14
15     for(const auto& z : F){
16         magnitudes.push_back(z.magnitude());
17     }
18
19     std::sort(magnitudes.begin(), magnitudes.end());
20
21     int keepCount = static_cast<int>(keepRatio * F.size

```

```

    ());
22     int thresholdIndex = static_cast<int>(F.size() -
        keepCount);
23
24     double threshold = magnitudes[thresholdIndex];
25
26     for(auto& z : F){
27         if(z.magnitude() < threshold){
28             z = Complex(0.0, 0.0);
29         }
30     }
31 }
32
33
34
35 int main() {
36     int width, height, channels;
37
38
39     // Nakon ovoga smo dobili niz koji sadrzi R G B R G
        B R G B ...
40     // za sve piksele slike redom
41     unsigned char* image = stbi_load(
42         "input.png",
43         &width,
44         &height,
45         &channels,
46         3
47     );
48
49     if (!image){
50         std::cout << "Greska pri učitavanju slike.\n";
51         return 1;
52     }
53
54     std::vector<Complex> R(width * height);
55     std::vector<Complex> G(width * height);
56     std::vector<Complex> B(width * height);
57
58     for(int i = 0; i < height; i++){

```

```

59         for(int j = 0; j < width; j++){
60             int rgbIndex = (i * width + j) * 3;
61             int index = i * width + j;
62
63             R[index] = Complex(image[rgbIndex], 0.0);
64             G[index] = Complex(image[rgbIndex + 1], 0.0)
65             ;
66             B[index] = Complex(image[rgbIndex + 2], 0.0)
67             ;
68         }
69     }
70
71     int paddedWidth = FFT::nextPowerOfTwo(width),
72         paddedHeight = FFT::nextPowerOfTwo(height);
73     std::vector<Complex> RFFT = FFT::fft2DPadded(R,width,
74         height,paddedWidth, paddedHeight);
75     std::vector<Complex> GFFT = FFT::fft2DPadded(G,width,
76         height,paddedWidth, paddedHeight);
77     std::vector<Complex> BFFT = FFT::fft2DPadded(B,width,
78         height,paddedWidth, paddedHeight);
79
80     keepLargestCoefficients(RFFT,0.05);
81     keepLargestCoefficients(GFFT,0.05);
82     keepLargestCoefficients(BFFT,0.05);
83
84     std::vector<unsigned char> output(width * height *
85         3);
86
87     std::vector<Complex> Rrec = FFT::ifft2DPadded(RFFT,
88         width,height,paddedWidth, paddedHeight);
89     std::vector<Complex> Grec = FFT::ifft2DPadded(GFFT,
90         width,height,paddedWidth, paddedHeight);
91     std::vector<Complex> Brec = FFT::ifft2DPadded(BFFT,
92         width,height,paddedWidth, paddedHeight);
93
94     for(int i = 0; i < width * height; i++){
95         double r = std::min(255.0, std::max(0.0, Rrec[i
96             ].real_part()));
97         double g = std::min(255.0, std::max(0.0, Grec[i

```



```

    ].real_part())));
88     double b = std::min(255.0, std::max(0.0, Brec[i
        ].real_part())));
89
90     output[i * 3] = static_cast<unsigned char>(r);
91     output[i * 3 + 1] = static_cast<unsigned char>(g
        );
92     output[i * 3 + 2] = static_cast<unsigned char>(b
        );
93 }
94
95     stbi_write_png("kompresovana.png", width,height,3,
        output.data(), width * 3);
96
97     stbi_image_free(image);
98
99     return 0;
100 }

```

Prednosti i ograničenja implementiranog algoritma

Implementirani algoritam ima nekoliko važnih prednosti. Prije svega, zasniva se na jasnoj matematičkoj ideji: slika se transformiše iz prostornog domena u frekvencijski domen, gdje je moguće analizirati značaj pojedinih frekvencijskih komponenti. Zadržavanjem koeficijenata najvećeg modula čuva se najveći dio vizuelno značajne informacije, dok se koeficijenti manjeg modula mogu odbaciti uz određeni gubitak kvaliteta.

Jedna od prednosti ovog pristupa je i jednostavna interpretacija rezultata. Smanjenjem procenta zadržanih koeficijenata može se direktno posmatrati kako se mijenja kvalitet rekonstruisane slike. Veći procenat zadržanih koeficijenata daje bolji kvalitet, dok manji procenat dovodi do jače kompresije, ali i do većeg gubitka detalja.

Dodatna prednost implementacije je to što se algoritam primjenjuje posebno na RGB kanale slike. Na taj način metoda nije ograničena samo na slike u nijansama sive, već se može koristiti i za slike u boji, kao i slike u drugim formatima.

Ipak implementirani algoritam ima određena ograničenja, a prvo se odnosi na vrijeme izvršavanja. Ručno implementirana rekurzivna FFT funkcija pravi dodatne vektore u svakom rekurzivnom pozivu, što dovodi do većeg broja kopiranja i alokacija memorije. Zbog toga program nije optimalan za veoma velike slike. Za praktične primjene nad velikim slikama efikasnije bi bilo koristiti optimizovane biblioteke kao što je FFTW ili implementirati iterativnu in-place verziju FFT algoritma. FFTW biblioteka nije uključena u ovom tekstu jer nije intuitivna za korišćenje, makar autoru.

Važno ograničenje odnosi se i na veličinu izlaznog fajla. U ovom radu se nakon filtriranja FFT koeficijenata primjenjuje inverzna FFT i rekonstruisana slika se čuva kao PNG fajl. Zbog toga smanjenje broja FFT koeficijenata ne mora nužno dovesti do manjeg fajla na disku. PNG format koristi sopstvenu kompresiju bez gubitaka, pa veličina izlaznog fajla zavisi i od strukture rekonstruisanih piksela. Prava kompresija u smislu veličine fajla zahtijevala bi čuvanje samo odabranih FFT koeficijenata i njihovih pozicija u posebnom formatu.

Takođe, odbacivanje koeficijenata manjeg modula može dovesti do замуćenja slike, gubitka sitnih detalja i pojave artefakata. Ovo je posebno

izraženo kod slika koje sadrže mnogo ivica, tekstura ili teksta. Zbog toga je potrebno pažljivo izabrati procenat koeficijenata koji se zadržava, kako bi se postigao prihvatljiv odnos između kompresije i kvaliteta slike.

Na osnovu ovih prednosti i ograničenja može se zaključiti da FFT kompresija predstavlja dobar eksperimentalni model za razumijevanje transformacije kompresije slike, ali da za praktičnu upotrebu zahtijeva dodatnu optimizaciju i poseban format za čuvanje kompresovanih podataka.